

JWT

JSON Web Token

Guida Completa - Dalle Basi alle Implementazioni Avanzate

Storia * Anatomia * Algoritmi * Implementazioni Python
Sicurezza * PASETO * OAuth 2.0 * Casi d'Uso Industriali

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici

Placeholder for table of contents.....0

1

Introduzione ai Token di Autenticazione

1. Introduzione ai Token di Autenticazione

1.1 Il Problema dell'Autenticazione Web

Prima di comprendere JWT, è fondamentale capire il problema che questa tecnologia si propone di risolvere. Il web nasce come sistema stateless: ogni richiesta HTTP è indipendente dalle precedenti, senza memoria dello stato. Questo è al tempo stesso un punto di forza (scalabilità) e una sfida (autenticazione).

Negli anni '90, quando il web commerciale muoveva i primi passi, i siti ricorrevano alle sessioni server-side: il server creava un record in memoria o database per ogni utente autenticato, assegnando un ID di sessione trasmesso al client tramite cookie. Ad ogni richiesta, il server consultava il proprio store per verificare la validità della sessione.

Questo approccio funzionava perfettamente su server singolo, ma con la diffusione delle architetture distribuite e dei cluster di server, emergevano problemi critici: se la sessione era memorizzata sul server A, le richieste indirizzate al server B non trovavano nulla. Le soluzioni adottate — sticky sessions o sessioni condivise su Redis/Memcached — aggiungevano complessità e punti di fallimento.

■ FONTI

- Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. UC Irvine.
- Rescorla, E. (2000). *HTTP Over TLS*. RFC 2818. IETF.

1.2 Da Cookie e Sessioni ai Token

La transizione verso le architetture a microservizi e le Single Page Application (SPA) nel periodo 2008-2014 ha accelerato la ricerca di un approccio alternativo. Il modello tradizionale mostrava i suoi limiti:

Scalabilità orizzontale: aggiungere nuovi nodi richiedeva sincronizzazione dello stato delle sessioni. Mobile-first: le app mobile non gestiscono i cookie con la stessa naturalezza dei browser. API-first: le REST API per definizione non dovrebbero mantenere stato server-side. Cross-domain: i cookie sono soggetti a Same-Origin Policy, complicando scenari multi-dominio.

L'idea alla base dei token è radicalmente diversa: invece di memorizzare lo stato sul server e consegnare al client solo un riferimento (ID sessione), si consegna al client tutto il necessario per provare la propria identità. Il server non deve mantenere nessuno stato: verifica semplicemente la firma del token ad ogni richiesta.

Questo paradigma viene chiamato "stateless authentication" e rappresenta uno dei fondamenti delle moderne architetture cloud-native. OAuth 2.0, apparso come RFC 6749 nel 2012, ha fornito il framework entro cui i token trovano la loro collocazione naturale.

■ FONTI

- Hardt, D. (2012). *The OAuth 2.0 Authorization Framework. RFC 6749. IETF.*
- Jones, M., et al. (2015). *JSON Web Token (JWT). RFC 7519. IETF.*

2. Storia e Origini di JWT

2.1 Le Radici: SAML e i Predecessori

La storia di JWT affonda le radici in SAML (Security Assertion Markup Language), uno standard XML-based per lo scambio di dati di autenticazione e autorizzazione tra parti, sviluppato da OASIS nel 2002 e portato alla versione 2.0 nel 2005.

SAML era potente ma verboso: un tipico assertion SAML poteva facilmente superare i 3KB, era basato su XML con firma XML Signature (xmldsig) — tecnologia solida ma complessa — e poco adatto alla leggerezza richiesta dalle applicazioni mobile o dai microservizi. L'header SAML in un'unica riga era già più lungo dell'intero payload di un JWT moderno.

Parallelamente, negli anni 2000 si affermava WS-Security nell'ecosistema SOAP, un'altra soluzione XML-based per autenticazione e autorizzazione nei web service enterprise. Anche qui, la complessità era il limite principale.

La comunità OpenID (2005) introdusse il concetto di identità federata semplificata, ma la prima versione usava ancora redirects e meccanismi complessi. OpenID Connect, la versione moderna che usa JWT nativamente, sarebbe arrivata solo nel 2014.

■ FONTI

- Cantor, S. et al. (2005). *Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0*. OASIS S
- Nadalin, A. et al. (2006). *Web Services Security: SOAP Message Security 1.1*. OASIS Standard.

2.2 La Nascita di JWT: 2010-2015

Il lavoro che ha portato a JWT inizia intorno al 2010 presso Google e altri grandi player tecnologici che stavano costruendo la prima generazione di API OAuth. La necessità era chiara: un formato token leggero, self-contained, verificabile, basato su JSON invece che su XML.

Michael B. Jones (Microsoft), John Bradley (Ping Identity) e Nat Sakimura (NRI) sono i principali autori del draft originale. Nel novembre 2010 viene pubblicato il primo Internet-Draft IETF per "JSON Web Token". Contestualmente nascevano gli standard correlati:

JSON Web Signature (JWS) — per la firma del payload. JSON Web Encryption (JWE) — per la cifratura del payload. JSON Web Key (JWK) — per la rappresentazione di chiavi crittografiche. JSON Web Algorithms (JWA) — per la definizione degli algoritmi supportati.

Questi quattro standard, insieme a JWT, formano la famiglia JOSE (JSON Object Signing and Encryption). Dopo anni di revisioni e feedback dalla community, nel maggio 2015 viene pubblicato l'RFC 7519 — lo standard definitivo JWT — insieme agli RFC 7515 (JWS), 7516 (JWE), 7517 (JWK) e 7518 (JWA).

■ FONTI

- Jones, M., Bradley, J., Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF.
- Jones, M., Bradley, J., Sakimura, N. (2015). *JSON Web Signature (JWS)*. RFC 7515. IETF.
- Jones, M., Hildebrand, J. (2015). *JSON Web Encryption (JWE)*. RFC 7516. IETF.

2.3 L'Adozione di Massa: 2015-2020

Dopo la pubblicazione degli RFC nel 2015, JWT ha conosciuto una crescita esplosiva. Diversi fattori hanno contribuito:

OpenID Connect 1.0 (2014) ha adottato JWT come formato nativo per i suoi ID Token, portando immediatamente JWT a milioni di applicazioni. Auth0, fondata nel 2013, ha costruito il suo intero business model intorno a JWT, contribuendo enormemente alla diffusione e alla documentazione dello standard. I principali framework web hanno aggiunto supporto nativo: Spring Security (Java), ASP.NET Core (C#), Django REST Framework (Python), Express.js (Node.js).

Nello stesso periodo sono emerse le prime vulnerabilità significative. Nel 2015 viene scoperta la vulnerabilità dell'algoritmo "none": poiché JWT permetteva di specificare alg=none per token non firmati, alcuni server accettavano token con firma rimossa ma alg impostato a none. Nel 2016 l'attacco di confusion tra algoritmi HS256/RS256: se un server si aspetta RS256 ma accetta anche HS256, un attaccante può firmare il token con la chiave pubblica del server (disponibile pubblicamente) usando HS256.

Queste vulnerabilità, ampiamente studiate e corrette nelle librerie moderne, hanno portato alla pubblicazione di best practices più rigorose e a una maggiore consapevolezza della community sulla corretta implementazione.

■ FONTI

- Sakimura, N. et al. (2014). *OpenID Connect Core 1.0*. OpenID Foundation.
- Auth0 Blog. (2015). *Critical vulnerabilities in JSON Web Token libraries*.
- McLean, T. (2015). *Critical Vulnerabilities in JWT*. Cybersecurity Report.

3

Anatomia di un JWT

3. Anatomia di un JWT

3.1 Struttura: Header, Payload, Signature

Un JWT è una stringa composta da tre parti separate da punti (.), ognuna codificata in Base64URL:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gR69lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

HEADER: Contiene il tipo di token e l'algoritmo di firma. Esempio decodificato: {"alg": "HS256", "typ": "JWT"}

PAYLOAD: Contiene i "claims" — le affermazioni sull'utente e metadati. Esempio decodificato: {"sub": "1234567890", "name": "John Doe", "iat": 1516239022}

SIGNATURE: È la firma crittografica calcolata su Header e Payload usando la chiave segreta (o privata). Per HMAC-SHA256: HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)

La codifica Base64URL è simile al Base64 standard ma usa - al posto di + e _ al posto di /, senza padding = finale. Questa scelta rende il token safe per l'uso in URL e header HTTP senza ulteriore encoding.

■ FONTI

- Jones, M. et al. (2015). *JSON Web Token (JWT). RFC 7519, Section 3. IETF.*

3.2 I Claims Standard (Registered Claims)

RFC 7519 definisce un insieme di "Registered Claims" — nomi di claim predefiniti con semantica standard. L'uso è raccomandato ma non obbligatorio:

iss (Issuer): Chi ha emesso il token. Es: "https://auth.example.com" **sub (Subject):** Il soggetto del token, tipicamente l'ID utente. Es: "user_12345" **aud (Audience):** Chi deve consumare il token. Può essere stringa o array. Es: "api.example.com" **exp (Expiration Time):** Timestamp Unix di scadenza. Il server deve rifiutare token scaduti. **nbf (Not Before):** Timestamp Unix prima del quale il token non è valido. **iat (Issued At):** Timestamp Unix di emissione. Utile per calcolare l'età del token. **jti (JWT ID):** Identificatore univoco del token. Utile per prevenire replay attacks.

Tutti i timestamp sono in secondi dall'epoch Unix (1 gennaio 1970 UTC), non millisecondi come in JavaScript. Questa è una fonte comune di bug: ricordate di dividere per 1000 i timestamp JS o di usare `int(time.time())` in Python.

■ FONTI

- Jones, M. et al. (2015). *JSON Web Token (JWT)*. RFC 7519, Section 4.1. IETF.

3.3 Claims Pubblici e Privati

Oltre ai Registered Claims, JWT supporta due altre categorie:

PUBLIC CLAIMS: Definiti dall'IANA JSON Web Token Claims Registry o con nomi URI per evitare collisioni. Esempi dall'IANA registry: email, name, phone_number, address, locale, picture, profile, roles, groups.

PRIVATE CLAIMS: Accordi tra produttore e consumatore del token, senza registrazione. Essendo privati, potrebbero collidere con altri sistemi. Best practice: usare namespace aziendali come prefisso. Es: "https://mycompany.com/roles": ["admin", "user"]

Un payload tipico di un'applicazione enterprise potrebbe contenere: { "iss": "https://auth.myapp.com", "sub": "user_98765", "aud": ["api.myapp.com", "admin.myapp.com"], "exp": 1735689600, "iat": 1735686000, "jti": "a1b2c3d4-e5f6-7890", "email": "mario@example.com", "https://myapp.com/roles": ["editor", "viewer"], "https://myapp.com/tenant": "acme-corp" }

■ FONTI

- IANA. (2023). *JSON Web Token Claims Registry*.
- Jones, M. et al. (2015). *JSON Web Token (JWT)*. RFC 7519, Section 4.2-4.3. IETF.

4. Algoritmi Crittografici in JWT

4.1 HMAC: Algoritmi Simmetrici (HS256, HS384, HS512)

Gli algoritmi HMAC (Hash-based Message Authentication Code) usano una singola chiave segreta condivisa sia per firmare che per verificare il token. Sono definiti in JWA (RFC 7518):

HS256: HMAC usando SHA-256. Output 256 bit. Chiave raccomandata ≥ 256 bit. HS384: HMAC usando SHA-384. Output 384 bit. Chiave raccomandata ≥ 384 bit. HS512: HMAC usando SHA-512. Output 512 bit. Chiave raccomandata ≥ 512 bit.

VANTAGGI: Velocissimi da calcolare, semplici da implementare, ideali quando emittente e verificatore sono la stessa entità (es: backend monolitico).

SVANTAGGI: La chiave segreta deve essere condivisa con TUTTI i servizi che verificano il token. In un'architettura a microservizi, se la chiave trapela da uno qualsiasi dei servizi, TUTTI i token sono compromessi. Non adatto per scenari in cui più parti indipendenti devono verificare token.

CASO D'USO IDEALE: Applicazione monolitica, servizio singolo, o scenario in cui emittente e verificatori sono completamente trusted e nello stesso perimetro di sicurezza.

■ FONTI

- Jones, M. et al. (2015). *JSON Web Algorithms (JWA)*. RFC 7518, Section 3.2. IETF.
- Krawczyk, H. et al. (1997). *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104. IETF.

4.2 RSA: Algoritmi Asimmetrici (RS256, RS384, RS512, PS256)

Gli algoritmi RSA usano una coppia di chiavi: privata per firmare, pubblica per verificare. La chiave pubblica può essere distribuita liberamente senza compromettere la sicurezza.

RS256: RSASSA-PKCS1-v1_5 con SHA-256. Chiave minima raccomandata: 2048 bit. RS384: RSASSA-PKCS1-v1_5 con SHA-384. Chiave minima raccomandata: 2048 bit. RS512: RSASSA-PKCS1-v1_5 con SHA-512. Chiave minima raccomandata: 2048 bit. PS256: RSASSA-PSS con SHA-256 e MGF1-SHA256. Schema di padding più moderno. PS384: RSASSA-PSS con SHA-384. PS512: RSASSA-PSS con SHA-512.

Il padding PSS (Probabilistic Signature Scheme) è considerato più sicuro del PKCS1-v1_5 ed è raccomandato nelle nuove implementazioni. La principale differenza è che PSS introduce randomness nella firma (ogni firma è diversa anche con stessi dati), mentre PKCS1-v1_5 produce sempre la stessa firma deterministica.

VANTAGGI: Distribuzione sicura della chiave di verifica; ideale per architetture distribuite; la compromissione di un servizio verificatore non compromette l'emittente.

SVANTAGGI: Più lenti di HMAC (5-10x); le chiavi sono più grandi; gestione della coppia di chiavi più complessa.

■ FONTI

- Jones, M. et al. (2015). *JSON Web Algorithms (JWA)*. RFC 7518, Section 3.3. IETF.
- Jonsson, J., Kaliski, B. (2003). *PKCS #1: RSA Cryptography Specifications Version 2.1*. RFC 3447. IETF.

4.3 ECDSA: Curve Ellittiche (ES256, ES384, ES512)

Gli algoritmi ECDSA (Elliptic Curve Digital Signature Algorithm) usano la crittografia a curve ellittiche — asimmetrici come RSA ma con chiavi molto più compatte e operazioni più veloci.

ES256: ECDSA con curva P-256 (secp256r1) e SHA-256. Chiave privata: 256 bit. ES384: ECDSA con curva P-384 (secp384r1) e SHA-384. Chiave privata: 384 bit. ES512: ECDSA con curva P-521 (secp521r1) e SHA-512. Chiave privata: 521 bit.

Confronto con RSA per sicurezza equivalente: RSA 2048 bit $\approx$ ECDSA P-256 (256 bit): stessa resistenza agli attacchi RSA 3072 bit $\approx$ ECDSA P-384 (384 bit) RSA 7680 bit $\approx$ ECDSA P-521 (521 bit)

Questo rende ECDSA molto attraente per dispositivi embedded, IoT, o applicazioni mobile dove la dimensione delle chiavi e la velocità delle operazioni crittografiche sono importanti. L'algoritmo EdDSA (ES256K, Ed25519) è una variante ancora più moderna usata in blockchain e sistemi crittografici avanzati, non ancora standard in JWT ma supportato da alcune librerie.

■ FONTI

- Jones, M. et al. (2015). *JSON Web Algorithms (JWA)*. RFC 7518, Section 3.4. IETF.
- NIST. (2013). *Digital Signature Standard (DSS)*. FIPS PUB 186-4.

5

Implementazione Python: Creare JWT

5. Implementazione Python: Creare JWT

5.1 Setup e Installazione

La libreria PyJWT è lo standard de-facto per JWT in Python. È mantenuta attivamente, segue gli RFC, e supporta tutti gli algoritmi principali.

Installazione:

```
pip install PyJWT[crypto]
```

Il flag [crypto] installa le dipendenze crittografiche (cryptography) necessarie per RSA ed ECDSA. Per solo HMAC è sufficiente `pip install PyJWT`.

Versioni raccomandate al 2026:

```
PyJWT >= 2.8.0 (breaking changes da 1.x a 2.x: cambia l'API)
cryptography >= 42.0.0
```

Verifica installazione:

```
import jwt
print(jwt.__version__) # Deve essere >= 2.8.0
```

Attenzione: esiste anche la libreria "python-jwt" (diversa da PyJWT) e "jose" (python-jose). Questa guida usa esclusivamente PyJWT, la più diffusa e mantenuta.

■ FONTI

- PyJWT Documentation. (2024). <https://pyjwt.readthedocs.io/>
- PyPI. (2024). PyJWT package statistics.

5.2 Creare JWT con HMAC (HS256)

Esempio completo di creazione token HMAC:

```
# === ESEMPIO 1: JWT Base con HS256 ===
import jwt
import time
from datetime import datetime, timedelta, timezone
```

SECRET_KEY = "chiave-super-segreta-minimo-32-caratteri-raccomandati"

```
def create_token(user_id: str, username: str, roles: list) -> str:
    now = datetime.now(timezone.utc)
    payload = {
```

```

"iss": "https://auth.myapp.com", # Issuer
"sub": user_id, # Subject
"aud": "api.myapp.com", # Audience
"exp": now + timedelta(hours=1), # Expiration
"nbf": now, # Not Before
"iat": now, # Issued At
"jti": f"token_{int(time.time())}", # JWT ID univoco
"username": username,
"roles": roles,
}
token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")
return token

# Uso:

```

```
token = create_token("user_123", "mario_rossi", ["admin", "editor"]) print(f"Token: {token[:50]}...")
```

```

# Decodifica il token (senza verifica per debug):
import base64, json

```

```

parts = token.split(".") header_decoded = json.loads(base64.b64decode(parts[0] + "=="))
payload_decoded = json.loads(base64.b64decode(parts[1] + "==")) print(f"Header:
{header_decoded}") print(f"Payload: {payload_decoded}")

```

■ FONTI

- PyJWT Documentation. (2024). Encoding & Decoding Tokens with HS256.

5.3 Creare JWT con RSA (RS256) e ECDSA (ES256)

Per RSA ed ECDSA è necessario generare prima la coppia di chiavi.

```

# === ESEMPIO 2: Generazione Chiavi RSA e JWT RS256 ===
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import jwt

# Genera coppia di chiavi RSA 2048 bit

```

```
private_key = rsa.generate_private_key(
```

```

    public_exponent=65537,
    key_size=2048,

```

```
) public_key = private_key.public_key()
```

```
# Serializza per salvare su file
```

```
private_pem = private_key.private_bytes(
```

```
encoding=serialization.Encoding.PEM,
format=serialization.PrivateFormat.TraditionalOpenSSL,
encryption_algorithm=serialization.NoEncryption()
```

```
) public_pem = public_key.public_bytes(
```

```
encoding=serialization.Encoding.PEM,
format=serialization.PublicFormat.SubjectPublicKeyInfo
```

```
)
```

```
# Salva le chiavi
```

```
with open("private.pem", "wb") as f:
```

```
f.write(private_pem)
```

```
with open("public.pem", "wb") as f:
```

```
f.write(public_pem)
```

```
# Crea token RS256
from datetime import datetime, timedelta, timezone
```

```
payload = {
```

```
"sub": "user_456",
"iss": "https://auth.example.com",
"exp": datetime.now(timezone.utc) + timedelta(hours=2),
"iat": datetime.now(timezone.utc),
```

```
} token_rs256 = jwt.encode(payload, private_key, algorithm="RS256")
```

```
# === ESEMPIO 3: JWT con ES256 (ECDSA P-256) ===
from cryptography.hazmat.primitives.asymmetric import ec
```

```
ec_private_key = ec.generate_private_key(ec.SECP256R1()) ec_public_key =
ec_private_key.public_key()
```

```
token_es256 = jwt.encode(payload, ec_private_key, algorithm="ES256") print(f"RS256 token length:
{len(token_rs256)} chars") print(f"ES256 token length: {len(token_es256)} chars")
```

```
# ECDSA produce token più compatti di RSA
```

■ FONTI

- PyJWT Documentation. (2024). RSA and EC Keys.
- Cryptography library docs. (2024). <https://cryptography.io/>

6

Implementazione Python: Validare JWT

6. Implementazione Python: Validare JWT

6.1 Validazione Base e Opzioni

La validazione di un JWT include: verifica della firma, verifica della scadenza (exp), verifica del not-before (nbf), e opzionalmente verifica di issuer e audience.

```
# === ESEMPIO 4: Validazione Completa JWT ===
import jwt
from jwt.exceptions import (
    ExpiredSignatureError,
    InvalidSignatureError,
    InvalidAudienceError,
    InvalidIssuerError,
    DecodeError,
```

)

SECRET_KEY = "chiave-super-segrete-minimo-32-caratteri-raccomandati"

```
def validate_token(token: str) -> dict | None:
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=["HS256"], # Lista ESPLICITA, mai lasciare vuoto
            audience="api.myapp.com", # Verifica audience
            issuer="https://auth.myapp.com", # Verifica issuer
            options={
                "require": ["exp", "iat", "sub"], # Claims obbligatori
                "verify_exp": True,
                "verify_iat": True,
                "verify_nbf": True,
            }
        )
        return payload

    except ExpiredSignatureError:
        print("Token scaduto")
    except InvalidSignatureError:
        print("Firma non valida")
    except InvalidAudienceError:
        print("Audience non valida")
    except InvalidIssuerError:
        print("Issuer non valido")
    except DecodeError as e:
        print(f"Token malformato: {e}")
```

```
return None

# Uso:
```

result = validate_token(token) if result:

```
print(f"Utente autenticato: {result['sub']}")
print(f"Ruoli: {result.get('roles', [])}")
```

■ FONTI

- *PyJWT Documentation. (2024). Decoding Tokens. Options.*
- *Jones, M. et al. (2015). JWT. RFC 7519, Section 7.2 Validating a JWT. IETF.*

6.2 Validazione con Chiave Pubblica RSA e JWKS

In produzione, la chiave pubblica può essere distribuita tramite JWKS (JSON Web Key Set), un endpoint HTTP che espone le chiavi pubbliche in formato JSON standardizzato.

```
# === ESEMPIO 5: Validazione con JWKS ===
import jwt
import requests
from jwt import PyJWKClient

# JWKS endpoint tipico di provider OAuth2 (Google, Auth0, etc.)
```

JWKS_URL = "https://www.googleapis.com/oauth2/v3/certs"

```
def validate_google_token(token: str, client_id: str) -> dict | None:
    jwks_client = PyJWKClient(JWKS_URL, cache_keys=True)
    try:
        # Recupera automaticamente la chiave dal JWKS
        signing_key = jwks_client.get_signing_key_from_jwt(token)
        payload = jwt.decode(
            token,
            signing_key.key,
            algorithms=["RS256"],
            audience=client_id,
        )
        return payload
    except Exception as e:
        print(f"Validazione fallita: {e}")
        return None

# === ESEMPIO 6: Creare e Servire JWKS proprio ===
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.backends import default_backend
import base64, json

def rsa_key_to_jwk(public_key, kid: str) -> dict:
    pub_numbers = public_key.public_key().public_numbers()
```



```
def to_base64url(n):
    length = (n.bit_length() + 7) // 8
    return base64.urlsafe_b64encode(
        n.to_bytes(length, 'big')
    ).rstrip(b'=').decode()
    return {
        "kty": "RSA",
        "use": "sig",
        "kid": kid,
        "alg": "RS256",
        "n": to_base64url(pub_numbers.n),
        "e": to_base64url(pub_numbers.e),
    }

# L'endpoint /.well-known/jwks.json deve restituire:
```

```
jwks = {"keys": [rsa_key_to_jwk(private_key, "key-2026-01")]} print(json.dumps(jwks, indent=2))
```

■ FONTI

- Jones, M. (2015). *JSON Web Key (JWK)*. RFC 7517. IETF.
- PyJWT Documentation. (2024). *PyJWKClient* class.

6.3 Middleware di Autenticazione con FastAPI

Un esempio pratico di middleware JWT integrato in FastAPI, il framework Python più diffuso per API REST.

```
# === ESEMPIO 7: FastAPI + JWT Middleware ===
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
from datetime import datetime, timedelta, timezone
from pydantic import BaseModel
```

```
app = FastAPI()
security = HTTPBearer()
SECRET_KEY = "produzione-usare-variabile-ambiente-mai-hardcoded"
ALGORITHM = "HS256"
```

```
class TokenData(BaseModel):
    user_id: str
    roles: list[str] = []

def create_access_token(user_id: str, roles: list) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": user_id,
        "roles": roles,
        "iat": now,
        "exp": now + timedelta(minutes=30), # Access token: vita breve
    }
```

```

return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(
credentials: HTTPAuthorizationCredentials = Depends(security)

```

) -> TokenData:

```

token = credentials.credentials
try:
payload = jwt.decode(
token, SECRET_KEY, algorithms=[ALGORITHM]
)
return TokenData(
user_id=payload["sub"],
roles=payload.get("roles", [])
)
except jwt.ExpiredSignatureError:
raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Token scaduto",
headers={"WWW-Authenticate": "Bearer"},
)
except jwt.InvalidTokenError:
raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Token non valido",
)

def require_role(role: str):
def checker(user: TokenData = Depends(get_current_user)):
if role not in user.roles:
raise HTTPException(
status_code=status.HTTP_403_FORBIDDEN,
detail=f"Ruolo '{role}' richiesto"
)
return user
return checker

@app.post("/login")
async def login(username: str, password: str):
# Verifica credenziali (semplificato)
if username == "admin" and password == "secret":
token = create_access_token("user_1", ["admin"])
return {"access_token": token, "token_type": "bearer"}
raise HTTPException(status_code=401, detail="Credenziali errate")

@app.get("/profile")
async def get_profile(user: TokenData = Depends(get_current_user)):
return {"user_id": user.user_id, "roles": user.roles}

@app.delete("/users/{user_id}")
async def delete_user(

```

```
user_id: str,  
admin: TokenData = Depends(require_role("admin"))
```

):

```
return {"message": f"Utente {user_id} eliminato da {admin.user_id}"}
```

■ FONTI

- *FastAPI Documentation*. (2024). OAuth2 with Password and JWT Tokens.
- Ramirez, S. (2024). *FastAPI Security*. <https://fastapi.tiangolo.com/tutorial/security/>

7

Refresh Token e Gestione del Ciclo di Vita

7. Refresh Token e Gestione del Ciclo di Vita

7.1 Access Token vs Refresh Token

Una delle scelte architetturali più importanti con JWT riguarda la durata del token. Due approcci opposti:

TOKEN LONGEVO (mala practice): Token con exp di 24h, 7 giorni o addirittura senza scadenza. Semplice da implementare ma pericoloso: se il token viene compromesso, l'attaccante ha accesso prolungato senza possibilità di revoca (JWT è stateless, non esiste blacklist nativa).

TOKEN BREVE + REFRESH (best practice): Access token con vita breve (15 minuti, 30 minuti al massimo), affiancato da un refresh token con vita lunga (7-30 giorni) conservato in modo sicuro. Quando l'access token scade, il client usa il refresh token per ottenerne uno nuovo senza re-autenticazione.

Il refresh token è tipicamente: - NON un JWT (o se JWT, con claim minimi) - Conservato nel database del server (per poterlo revocare) - Consegnato solo via cookie HttpOnly Secure (non accessibile a JavaScript) - Soggetto a "rotation": ad ogni refresh, viene invalidato e sostituito con uno nuovo

■ FONTI

- Auth0. (2023). *Refresh Token Rotation*.
- IETF. (2023). *OAuth 2.0 Security Best Current Practice*. RFC 9700.

7.2 Implementazione Refresh Token in Python

```
# === ESEMPIO 8: Sistema Access + Refresh Token Completo ===
import jwt, secrets, hashlib
from datetime import datetime, timedelta, timezone
from dataclasses import dataclass, field
from typing import Optional
```

```
SECRET_KEY = "access-token-secret-key"
REFRESH_SECRET = "refresh-token-secret-different-key"
```

```
# In produzione: database Redis o PostgreSQL
```

```
refresh_token_store: dict[str, dict] = {}
```

```
@dataclass
class AuthTokens:
```

```

access_token: str
refresh_token: str
access_expires_in: int # secondi
token_type: str = "bearer"

def create_access_token(user_id: str, roles: list) -> str:
    now = datetime.now(timezone.utc)
    return jwt.encode({
        "sub": user_id, "roles": roles,
        "iat": now,
        "exp": now + timedelta(minutes=15), # Vita breve!
        "type": "access",
    }, SECRET_KEY, algorithm="HS256")

def create_refresh_token(user_id: str) -> str:
    # Refresh token: stringa random + JWT firmato separatamente
    jti = secrets.token_urlsafe(32) # ID univoco
    now = datetime.now(timezone.utc)
    token = jwt.encode({
        "sub": user_id, "jti": jti,
        "iat": now,
        "exp": now + timedelta(days=30),
        "type": "refresh",
    }, REFRESH_SECRET, algorithm="HS256")

    # Salva hash del token nel database (mai il token in chiaro)
    token_hash = hashlib.sha256(token.encode()).hexdigest()
    refresh_token_store[token_hash] = {
        "user_id": user_id, "jti": jti,
        "created_at": now.isoformat(),
        "used": False
    }
    return token

def login(user_id: str, roles: list) -> AuthTokens:
    return AuthTokens(
        access_token=create_access_token(user_id, roles),
        refresh_token=create_refresh_token(user_id),
        access_expires_in=900 # 15 minuti
    )

def refresh_tokens(refresh_token: str) -> Optional[AuthTokens]:
    try:
        payload = jwt.decode(
            refresh_token, REFRESH_SECRET, algorithms=["HS256"]
        )
        if payload.get("type") != "refresh":
            return None
    
```

```

# Verifica che il token non sia già stato usato (rotation)
token_hash = hashlib.sha256(refresh_token.encode()).hexdigest()
    
```

```
stored = refresh_token_store.get(token_hash)

if not stored or stored["used"]:
    # Possibile replay attack! Invalida tutto
    print("ATTENZIONE: Refresh token riusato!")
    return None

# Marca il vecchio token come usato (rotation)
stored["used"] = True

# Emette nuova coppia di token
user_id = payload["sub"]
roles = ["user"] # Ricarica dal database
return login(user_id, roles)

except jwt.InvalidTokenError:
    return None

# Test del flusso completo
```

```
tokens = login("user_99", ["editor"]) print(f'Access token (15 min): {tokens.access_token[:40]}...')
new_tokens = refresh_tokens(tokens.refresh_token) print(f'Nuovo access token: {new_tokens.access_token[:40]}...')
```

■ FONTI

- Auth0. (2023). *Refresh Token Rotation Best Practices*.
- OWASP. (2024). *JSON Web Token Cheat Sheet*.

8

Vulnerabilità e Sicurezza

8. Vulnerabilità e Sicurezza

8.1 Vulnerabilità Storiche e Come Evitarle

JWT ha avuto diverse vulnerabilità significative nelle sue implementazioni. Conoscerle è fondamentale per scrivere codice sicuro.

1. ALGORITMO NONE (2015): Alcune librerie accettavano alg: "none" saltando la verifica della firma. Mitigazione: Specifica sempre la lista esplicita di algoritmi accettati nel decoder. Non fidarti dell'header del token per scegliere l'algoritmo.

2. CONFUSION RS256/HS256 (2016): Se un server si aspetta RS256 ma accetta anche HS256, un attaccante può firmare con la chiave pubblica (nota) usando HS256. Mitigazione: Usa una lista WHITELIST di algoritmi. Mai accettare l'algoritmo proposto dal token senza validazione.

3. JWT FORGING con JWKS (2019): Se il server segue il parametro "jku" o "x5u" nell'header senza validazione, un attaccante può puntare a un JWKS malevolo. Mitigazione: Ignora "jku", "x5u", "x5c" negli header. Usa solo chiavi/JWKS configurati staticamente.

4. PAYLOAD INJECTION nel KID: Il parametro "kid" (Key ID) nell'header veniva usato direttamente in query SQL o come path su filesystem in alcune implementazioni. Mitigazione: Validare e sanificare sempre il kid. Usare una mappa fissa di kid -> chiave.

5. TIMING ATTACKS sulla firma: Confronto naive delle firme (==) è vulnerabile ad attacchi timing. Mitigazione: Le librerie moderne usano hmac.compare_digest() che è time-constant.

■ FONTI

- Auth0. (2015). *Critical vulnerabilities in JSON Web Token libraries*.
- OWASP. (2024). *JWT Security Cheat Sheet*.
- CVE-2022-21449: *ECDSA signature bypass in Java 15-18*.

8.2 Dove Conservare i JWT nel Browser

La domanda più dibattuta nella community JWT: dove conservare il token nel client?

OPZIONE 1 - localStorage/sessionStorage: Facile, persiste tra sessioni, accessibile da JavaScript. PROBLEMA: Vulnerabile a XSS (Cross-Site Scripting). Se un attaccante inietta JavaScript nella pagina, legge tutti i token.

OPZIONE 2 - Cookie HttpOnly Secure SameSite=Strict: Non accessibile a JavaScript (protetto da XSS), trasmesso automaticamente via HTTPS. PROBLEMA: Vulnerabile a CSRF se non configurato

correttamente. La combinazione SameSite=Strict + token CSRF mitiga il rischio.

RACCOMANDAZIONE OWASP (2024): Per applicazioni web tradizionali, usare cookie HttpOnly Secure SameSite=Strict. Per SPA che devono gestire più domini, considerare memory-only (variabile JavaScript in-memory, persa al reload) con refresh token in cookie HttpOnly.

CASO MOBILE/IoT: Secure storage del sistema operativo (Keychain iOS, Keystore Android). Mai hardcodare nei sorgenti o in SharedPreferences/UserDefaults non cifrate.

```
# === ESEMPIO 9: Cookie sicuro con FastAPI ===
from fastapi import Response

@app.post('/login-cookie')
async def login_with_cookie(response: Response):
    token = create_access_token('user_1', ['user'])
    response.set_cookie(
        key='access_token',
        value=token,
        httponly=True, # Non accessibile da JS
        secure=True, # Solo HTTPS
        samesite='strict', # Protegge da CSRF
        max_age=900, # 15 minuti
        path='/api', # Limita scope
    )
    return {'message': 'Login effettuato'}
```

■ FONTI

- OWASP. (2024). *JSON Web Token for Java Cheat Sheet*.
- MDN Web Docs. (2024). *Set-Cookie: HttpOnly, Secure, SameSite*.

9. JWT vs Alternative: Confronto con Tecnologie Concorrenti

9.1 JWT vs PASETO (Platform-Agnostic Security Tokens)

PASETO (Platform-Agnostic Security Tokens) nasce nel 2018 come risposta critica alle vulnerabilità storiche di JWT. L'idea fondamentale è eliminare la scelta crittografica dall'implementatore: PASETO non permette di scegliere l'algoritmo, ogni versione ne usa uno specifico e sicuro.

Versioni PASETO: v1 (legacy): HMAC-SHA384 (local), RSA-PSS-2048-SHA384 (public). Per ambienti FIPS. v2: XSalsa20-Poly1305 (local), Ed25519 (public). Moderno e veloce. v3: AES-256-CTR + HMAC-SHA384 (local), ECDSA P-384 (public). FIPS-compliant moderno. v4: XChaCha20-Poly1305 (local), Ed25519 (public). Raccomandato per nuove implementazioni.

VANTAGGI di PASETO su JWT: - Nessuna scelta dell'algoritmo: impossibile "algorithm confusion" - Separazione esplicita tra local (simmetrico, cifrato) e public (asimmetrico, firmato) - Nessun alg=none possibile - Include "purpose" nel token prevenendo confusion attacks

SVANTAGGI di PASETO: - Ecosistema più limitato (meno librerie, meno supporto nei framework) - Non supportato nativamente da provider OAuth2 standard - Meno diffuso = meno revisione da parte della community

```
# pip install python-paseto
import paseto
from paseto.keys.symmetric_key import SymmetricKey
from paseto.protocols.v4 import ProtocolVersion4
```

```
key = SymmetricKey.generate(ProtocolVersion4) token = paseto.encode({"user": "mario"}, key)
payload = paseto.decode(token, key)
```

■ FONTI

- Arciszewski, S., Paragon Initiative Enterprises. (2018). PASETO Specification.
- IETF Draft. (2023). PASETO: Platform-Agnostic Security Tokens.

9.2 JWT vs Macaroons

Macaroons sono un sistema di token creato da Google nel 2014, basato su catene HMAC (chained HMAC). L'idea innovativa è la "decentralized authorization with attenuation": partendo da un token root, chiunque può creare token derivati con permessi più restrittivi (attenuazione), senza coinvolgere il server originale.

MECCANISMO: Un Macaroon è una catena di caveat (condizioni): root_token -> aggiungi "user = mario" -> aggiungi "time < 2026-12-31" -> firma

Ogni attenuazione aggiunge un caveat verificato da chi riceve il token. Non è possibile rimuovere caveat, solo aggiungerne.

VANTAGGI rispetto a JWT: - Attenuazione: puoi delegare permessi ristretti senza server round-trip - Third-party caveats: condizioni verificate da terze parti (es: "verifica 2FA su sito X") - Composability naturale per sistemi distribuiti

SVANTAGGI: - Complessità concettuale maggiore - Ecosistema molto limitato - Praticamente assenti nei framework mainstream - Revoca ancora complessa

USO REALE: Usati internamente da Cloudflare, alcune piattaforme storage. Non adatti come sostituto drop-in di JWT per autenticazione standard.

```
# pip install pymacaroons
from pymacaroons import Macaroon, Verifier
```

```
m = Macaroon(location='https://example.com', identifier='user123', key='secret')
m.add_first_party_caveat('user = mario')
m.add_first_party_caveat('role = editor')
print(m.serialize())
```

■ FONTI

- Birgisson, A. et al. (2014). *Macaroons: Cookies with Contextual Caveats*. Google.
- Cloudflare Blog. (2022). *Using Macaroons for API authentication*.

9.3 JWT vs SAML vs Session Cookie: Tabella Comparativa

Confronto diretto tra le principali tecnologie di autenticazione:

DIMENSIONE TOKEN: - Session Cookie: ~50 byte (solo ID) - JWT HS256: ~200-500 byte - JWT RS256: ~500-1000 byte - SAML 2.0: 3000-10000 byte - PASETO v4: ~200-400 byte

STATEFUL vs STATELESS: - Session Cookie: Stateful (server mantiene sessione) - JWT: Stateless (self-contained) - SAML: Stateful/Stateless (dipende da implementazione)

REVOCA: - Session Cookie: Immediata (cancella sessione dal server) - JWT: Complessa (blacklist, short exp + refresh rotation) - SAML: Dipende dall'implementazione

LINGUAGGI/Framework: - Session Cookie: Universale - JWT: Universale, librerie per ogni linguaggio - SAML: Principalmente Java/.NET enterprise

BEST FOR: - Session Cookie: App web tradizionale, monolite - JWT: API REST, microservizi, mobile, SPA - SAML: SSO enterprise, integrazione legacy - PASETO: Alta sicurezza, nuovi sistemi, no legacy constraint

■ FONTI

- OWASP. (2024). *Session Management Cheat Sheet*.
- Auth0 Blog. (2023). *JWT vs SAML vs Session Token comparison*.

10

Scenari Avanzati e Casi d'Uso Reali

10. Scenari Avanzati e Casi d'Uso Reali

10.1 JWT in Architetture Microservizi

Nei microservizi, JWT brilla per la sua natura stateless: ogni servizio verifica autonomamente il token senza dipendere da un servizio di sessione centralizzato. Tuttavia, ci sono pattern architetturali da seguire.

PATTERN 1 - API GATEWAY VALIDATION: L'API Gateway verifica il JWT e passa claims validati ai servizi interni come header: X-User-Id: user_123 X-User-Roles: admin,editor I servizi interni si fidano di questi header (comunicazione interna trusted), non rifiutano il JWT.

PATTERN 2 - SERVICE-TO-SERVICE JWT: Ogni microservizio ha il proprio JWT per autenticarsi verso altri servizi. Utile per mutual authentication in zero-trust networks.

```
# === ESEMPIO 10: Service-to-Service JWT ===
import jwt
from datetime import datetime, timedelta, timezone

def create_service_token(service_name: str, target_service: str) -> str:
    now = datetime.now(timezone.utc)
    return jwt.encode({
        "iss": f"service:{service_name}",
        "aud": f"service:{target_service}",
        "sub": service_name,
        "iat": now,
        "exp": now + timedelta(minutes=5), # Token brevissimi S2S
        "type": "service",
        "permissions": ["read", "write"],
    }, SERVICE_SECRET_KEY, algorithm="HS256")

# Robot YASKAWA che chiama API del sistema di trattamento acque:
```

```
yaskawa_token = create_service_token(
```

```
    service_name="yaskawa-hc10-controller",
    target_service="water-treatment-api"
```

```
)
```

■ FONTI

- Newman, S. (2021). *Building Microservices*, 2nd Edition. O'Reilly.
- Richardson, C. (2018). *Microservices Patterns*. Manning.

10.2 JWT e WebSocket: Autenticazione Real-Time

I WebSocket presentano una sfida unica: la connessione HTTP iniziale può trasportare il JWT, ma i messaggi successivi sono binari/testo puri. Tre approcci:

APPROCCIO 1 - JWT nell'URL (sconsigliato): `ws://example.com/ws?token=eyJ...` Il token appare nei log del server ed è esposto nella history del browser.

APPROCCIO 2 - JWT nel primo messaggio (raccomandato): Dopo la connessione WebSocket, il client invia immediatamente un messaggio di tipo "auth" con il JWT. Il server non processa altri messaggi finché l'autenticazione non è completata.

APPROCCIO 3 - Cookie HttpOnly per l'handshake iniziale (best per browser): Il JWT è in un cookie HttpOnly. Durante l'upgrade WebSocket, il browser include automaticamente il cookie nell'header della richiesta HTTP di upgrade.

```
# === ESEMPIO 11: WebSocket con JWT (usando websockets) ===
import asyncio, websockets, json, jwt

async def ws_handler(websocket):
    # Attendi messaggio di autenticazione
    try:
        auth_msg = await asyncio.wait_for(
            websocket.recv(), timeout=5.0
        )
        data = json.loads(auth_msg)

        if data.get("type") != "auth":
            await websocket.close(4001, "Auth required")
            return

        payload = jwt.decode(
            data["token"], SECRET_KEY, algorithms=["HS256"]
        )
        user_id = payload["sub"]
        print(f"WebSocket autenticato: {user_id}")

        # Ora la connessione è autenticata
        async for message in websocket:
            await websocket.send(
                json.dumps({"echo": message, "from": user_id})
            )

    except asyncio.TimeoutError:
        await websocket.close(4001, "Auth timeout")
    except jwt.InvalidTokenError:
        await websocket.close(4003, "Invalid token")
```

■ FONTI

- Fette, I., Melnikov, A. (2011). *The WebSocket Protocol. RFC 6455. IETF.*
- OWASP. (2024). *HTML5 Security Cheat Sheet - WebSockets.*

10.3 JWT Claims Personalizzati per Sistemi Industriali

Nel contesto industriale (automazione, PLC, SCADA, robot), JWT può essere usato per autenticare operatori, macchine e sessioni di controllo remoto con claim specifici al dominio.

```
# === ESEMPIO 12: JWT per Sistema di Automazione Industriale ===
import jwt
from datetime import datetime, timedelta, timezone
from enum import Enum

class OperatorLevel(str, Enum):
    VIEWER = "viewer" # Solo lettura
    OPERATOR = "operator" # Controllo base
    ENGINEER = "engineer" # Configurazione
    ADMIN = "admin" # Accesso completo

def create_industrial_token(
    operator_id: str,
    operator_level: OperatorLevel,
    allowed_cells: list[str], # Celle robotizzate accessibili
    allowed_machines: list[str], # Macchine specifiche
    station_id: str, # Stazione di login
```

) -> str:

```
now = datetime.now(timezone.utc)

# Token breve per sicurezza: 2 ore max per turno
exp_delta = timedelta(hours=2)
if operator_level == OperatorLevel.VIEWER:
    exp_delta = timedelta(hours=8)

return jwt.encode({
    # Standard claims
    "iss": "https://scada.plant.local",
    "sub": operator_id,
    "aud": "robot-control-system",
    "exp": now + exp_delta,
    "iat": now,
    "jti": f"{operator_id}_{int(now.timestamp())}",
    # Custom claims industriali
    "https://plant.local/level": operator_level.value,
    "https://plant.local/cells": allowed_cells,
    "https://plant.local/machines": allowed_machines,
    "https://plant.local/station": station_id,
    "https://plant.local/shift": get_current_shift(),
    "https://plant.local/emergency_stop": True, # Sempre abilitato
```

```

}, SCADA_PRIVATE_KEY, algorithm="RS256")

def get_current_shift() -> str:
    hour = datetime.now().hour
    if 6 <= hour < 14: return "morning"
    if 14 <= hour < 22: return "afternoon"
    return "night"

# Uso per operatore Yaskawa:

```

```
token = create_industrial_token(
```

```

    operator_id="op_carmelo_001",
    operator_level=OperatorLevel.ENGINEER,
    allowed_cells=["cell_A", "cell_B", "welding_zone"],
    allowed_machines=["HC10DTP-A01", "HC10DTP-A02", "GP7-B01"],
    station_id="HMI-LINE1-POS3",

```

```
)
```

■ FONTI

- IEC 62443. (2018). *Security for Industrial Automation and Control Systems*.
- NIST SP 800-82. (2023). *Guide to Operational Technology (OT) Security*.

10.4 Revoca JWT: Blacklist e Short-Lived Tokens

La revoca è il tallone d'Achille di JWT: essendo stateless, un token valido ma compromesso rimarrà valido fino alla scadenza. Le soluzioni principali:

APPROCCIO 1 - Token brevissimi (15 min): Accetta che la revoca non sia immediata, ma il danno è limitato alla breve finestra temporale.

APPROCCIO 2 - Blacklist con Redis: Mantieni un set Redis dei jti (JWT ID) revocati. Ad ogni richiesta, verifica che il jti non sia in blacklist. Aggiunge latenza ma permette revoca immediata.

APPROCCIO 3 - Token Version nel database: Ogni utente ha un token_version nel database. Il JWT include tale versione. Quando si fa logout o cambio password, incrementa la versione. Token con versione vecchia sono rifiutati.

```

# === ESEMPIO 13: Blacklist JWT con Redis ===
import redis, jwt
from datetime import datetime, timezone, timedelta

```

```
r = redis.Redis(host='localhost', port=6379, db=0)
```

```

def revoke_token(token: str, secret: str) -> bool:
    try:
        payload = jwt.decode(
            token, secret, algorithms=["HS256"],

```

```
options={"verify_exp": False} # Decodifica anche se scaduto
)
jti = payload.get("jti")
exp = payload.get("exp")
if not jti or not exp:
    return False

# Calcola TTL: mantieni in blacklist solo fino alla scadenza naturale
now = int(datetime.now(timezone.utc).timestamp())
ttl = max(exp - now, 0)

if ttl > 0:
    r.setex(f"blacklist:{jti}", ttl, "revoked")
    return True

except jwt.InvalidTokenError:
    return False

def is_token_revoked(token: str, secret: str) -> bool:
    try:
        payload = jwt.decode(token, secret, algorithms=["HS256"])
        jti = payload.get("jti")
        return r.exists(f"blacklist:{jti}") > 0
    except jwt.InvalidTokenError:
        return True # Token invalido = consideralo revocato

# Logout: revoca il token corrente
```

`revoke_token(current_user_token, SECRET_KEY)`

■ FONTI

- OWASP. (2024). *JWT Cheat Sheet - Token Revocation*.
- Auth0 Blog. (2023). *Revoking JWTs: Why and How*.

11

OpenID Connect e OAuth 2.0 con JWT

11. OpenID Connect e OAuth 2.0 con JWT

11.1 Il Ruolo di JWT in OAuth 2.0

OAuth 2.0 (RFC 6749, 2012) è un framework di autorizzazione, non di autenticazione. Definisce come un'applicazione (client) ottiene accesso a risorse per conto di un utente, senza che l'utente condivida le proprie credenziali. JWT è usato come formato degli Access Token e dei token opachi possono essere sostituiti da JWT trasparenti.

I principali grant type OAuth 2.0: - Authorization Code: Per web app con backend. Il più sicuro. - Authorization Code + PKCE: Per SPA e app mobile. Standard attuale. - Client Credentials: Per machine-to-machine (M2M). - Device Authorization: Per dispositivi senza browser (IoT, Smart TV).

PKCE (Proof Key for Code Exchange, RFC 7636) è ora obbligatorio per tutti i client pubblici: genera un code_verifier segreto, ne calcola il code_challenge (SHA-256), e include il challenge nella richiesta di autorizzazione. Il server verifica il verifier al momento dello scambio del codice, prevenendo intercettazione del codice di autorizzazione.

```
# === ESEMPIO 14: OAuth2 con PKCE in Python ===
import secrets, hashlib, base64

def generate_pkce_pair() -> tuple[str, str]:
    code_verifier = secrets.token_urlsafe(64) # 43-128 char
    digest = hashlib.sha256(code_verifier.encode()).digest()
    code_challenge = base64.urlsafe_b64encode(digest).rstrip(b'=').decode()
    return code_verifier, code_challenge
```

```
verifier, challenge = generate_pkce_pair() print(f"Verifier: {verifier}") print(f"Challenge: {challenge}")
```

```
# Challenge viene inviato al server OAuth, verifier viene conservato dal client
```

■ FONTI

- Hardt, D. (2012). *The OAuth 2.0 Authorization Framework. RFC 6749. IETF.*
- Sakimura, N. et al. (2015). *Proof Key for Code Exchange by OAuth Public Clients. RFC 7636. IETF.*

11.2 OpenID Connect: ID Token JWT

OpenID Connect (OIDC) 1.0 è il layer di autenticazione costruito sopra OAuth 2.0. Aggiunge il concetto di ID Token — un JWT contenente informazioni sull'utente autenticato — e standardizza claim come sub, email, name, picture.

I claim standard OIDC: sub: Identificatore univoco e stabile dell'utente presso l'IdP name: Nome completo given_name, family_name: Nome e cognome separati email: Indirizzo email email_verified: Boolean, email verificata dall'IdP phone_number: Numero di telefono picture: URL avatar locale: Preferenza di lingua zoneinfo: Timezone updated_at: Quando il profilo è stato aggiornato

Differenza fondamentale tra Access Token e ID Token in OIDC: - ID Token: Chi è l'utente (autenticazione). Consumato dal CLIENT. - Access Token: Cosa può fare l'utente (autorizzazione). Presentato alle API.

```
# === ESEMPIO 15: Verifica ID Token Google con PyJWT ===
import jwt, requests
from jwt import PyJWKClient
```

```
GOOGLE_JWKS = "https://www.googleapis.com/oauth2/v3/certs"  GOOGLE_CLIENT_ID =
"YOUR_CLIENT_ID.apps.googleusercontent.com"
```

```
def verify_google_id_token(id_token: str) -> dict | None:
    jwks_client = PyJWKClient(GOOGLE_JWKS, cache_keys=True)
    try:
        signing_key = jwks_client.get_signing_key_from_jwt(id_token)
        payload = jwt.decode(
            id_token,
            signing_key.key,
            algorithms=["RS256"],
            audience=GOOGLE_CLIENT_ID,
            issuer=["https://accounts.google.com",
                  "accounts.google.com"],
        )
        assert payload.get("email_verified") == True
        return payload
    except Exception as e:
        print(f"ID Token non valido: {e}")
        return None
```

■ FONTI

- Sakimura, N. et al. (2014). *OpenID Connect Core 1.0*. OpenID Foundation.
- Google Identity. (2024). *Verify the Google ID token on your server side*.

12. Best Practices, Checklist e Futuro di JWT

12.1 JWT Security Checklist Completa

Lista di controllo per implementazioni JWT sicure:

ALGORITMI: ■ Usa lista whitelist esplicita di algoritmi accettati ■ Non accettare mai alg=none ■ Preferisci RS256/ES256 per microservizi ■ Usa HS256 solo quando emittente e verificatore sono la stessa entità ■ Chiavi HMAC: minimo 256 bit (32 byte) di entropia reale

CLAIMS: ■ Imposta sempre exp con vita breve (max 1 ora per access token) ■ Usa sempre jti per permettere blacklisting ■ Verifica sempre aud per prevenire token passthrough attacks ■ Verifica sempre iss ■ Includi nbf se necessario

STORAGE CLIENT: ■ Browser: Cookie HttpOnly Secure SameSite=Strict ■ Mobile: OS Secure Storage (Keychain/Keystore) ■ Mai localStorage per token sensibili ■ Mai URL query parameter

TRASPORTO: ■ Solo HTTPS/TLS 1.2+ ■ Header Authorization: Bearer <token> ■ Refresh token solo via cookie HttpOnly

REVOCA: ■ Implementa refresh token rotation ■ Usa jti + blacklist Redis per revoca immediata (se necessario) ■ Cambia password = invalida tutti i refresh token

CHIAVI: ■ Ruota le chiavi regolarmente (almeno annuale) ■ Usa kid nell'header per supportare multiple chiavi ■ Mai hardcode chiavi nel codice sorgente ■ Usa variabili d'ambiente o secret manager (Vault, AWS Secrets Manager)

■ FONTI

- OWASP. (2024). *JSON Web Token Cheat Sheet for Java*.
- NIST SP 800-63B. (2020). *Digital Identity Guidelines: Authentication*.
- Auth0. (2024). *JWT Security Best Practices*.

12.2 Il Futuro: JWT, DPoP e Selective Disclosure

L'evoluzione di JWT continua con nuove estensioni che affrontano limiti attuali:

DPoP (Demonstrating Proof of Possession, RFC 9449, 2023): Lega un access token alla chiave crittografica del client. Previene l'uso del token rubato da un attaccante diverso dal legittimo possessore, poiché il client deve dimostrare il possesso della chiave privata ad ogni richiesta.

SD-JWT (Selective Disclosure JWT, IETF Draft 2024): Permette all'utente di rivelare solo alcuni claim dal token, preservando la privacy. Esempio: un token con età, nome, indirizzo — l'utente può presentare SOLO "maggiore di 18" senza rivelare data di nascita esatta.

JWT per Verifiable Credentials (W3C): Nel contesto dell'identità digitale decentralizzata (DID - Decentralized Identifiers), JWT viene usato come formato per Verifiable Credentials — credenziali digitali verificabili senza dipendere da un'autorità centrale.

eIDAS 2.0 e EUDIW: Il portafoglio di identità digitale europeo (EU Digital Identity Wallet), in sviluppo dal 2023, usa SD-JWT come formato base per le credenziali digitali dei cittadini europei.

```
# Esempio SD-JWT concettuale (libreria python-sd-jwt):  
from sd_jwt.issuer import SDJWTIssuer
```

```
claims = {
```

```
"sub": "user_123",  
"name": "Mario Rossi", # Selectively disclosable  
"birthdate": "1985-03-15", # Selectively disclosable  
"over_18": True, # Selectively disclosable  
"email": "mario@example.com", # Selectively disclosable
```

```
}
```

```
# L'utente può scegliere di rivelare solo: name + over_18  
# Nascondendo: birthdate, email
```

■ FONTI

- Fett, D. et al. (2023). *Demonstrating Proof of Possession (DPoP)*. RFC 9449. IETF.
- Fett, D. et al. (2024). *Selective Disclosure for JWTs (SD-JWT)*. IETF Draft.
- European Commission. (2023). *EU Digital Identity Wallet Architecture*.

A

Appendice — Risorse Utili e Strumenti

Appendice — Risorse Utili e Strumenti

Risorsa	URL / Riferimento
RFC 7519 — JWT Standard	https://datatracker.ietf.org/doc/html/rfc7519
RFC 7515 — JWS Standard	https://datatracker.ietf.org/doc/html/rfc7515
RFC 7517 — JWK Standard	https://datatracker.ietf.org/doc/html/rfc7517
RFC 7518 — JWA Standard	https://datatracker.ietf.org/doc/html/rfc7518
RFC 6749 — OAuth 2.0	https://datatracker.ietf.org/doc/html/rfc6749
OpenID Connect Core 1.0	https://openid.net/specs/openid-connect-core-1_0.html
PyJWT Documentation	https://pyjwt.readthedocs.io/
OWASP JWT Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
jwt.io — Debugger Online	https://jwt.io
PASETO Standard	https://paseto.io/
IANA JWT Claims Registry	https://www.iana.org/assignments/jwt/jwt.xhtml
DPoP RFC 9449	https://datatracker.ietf.org/doc/html/rfc9449